

Single-Cycle RISC-V Processor Project Report

EE/CS 371L/330L – Computer Architecture

Syed Faraz Ahmed Shah sf10162

Ryann Amir Alvi ra8019

Spring 2026 · Habib University

Repository: github.com/faraz18001/Computer-Arch-Labs

1. Processor Summary and Synthesis

We implemented a single-cycle RV32I RISC-V processor in Verilog, synthesized and deployed on the Digilent Basys3 FPGA (Artix-7 XC7A35T). The processor executes one instruction per clock cycle with the following datapath modules:

Module	Function
ProgramCounter	32-bit PC register with synchronous reset
InstructionMemory	256-entry ROM, word-addressed via <code>address[9:2]</code>
RegisterFile	32×32-bit register file; x0 hardwired to zero
ALU	Supports ADD, SUB, AND, OR, XOR, SLT
main_control	Generates 12 control signals from the 7-bit opcode
alu_control	Derives 4-bit ALU operation from <code>ALUOp</code> , <code>funct3</code> , and <code>funct7</code>
immGen	Decodes I/S/B/U/J-type immediates
DataMemory	512-entry RAM with memory-mapped I/O at address 0x100
branchAdder	Computes branch/jump target: $PC + (imm \ll 1)$

FPGA I/O Mapping. The 7-segment display shows either the Program Counter or the ALU result (selected via `sw[0]`). In Task B, 13 control signals are routed to the 16 onboard LEDs for real-time visualization: `RegWrite`, `MemRead`, `MemWrite`, `ALUSrc`, `MemtoReg`, `Branch`, `ALUSrcA`, `ALUOp[1:0]`, `PCSrc`, `Zero`, `Jump`, and `JumpReg`.

Clock. A clock divider derives a slow clock from the 100 MHz board clock. Task B uses `clk_div[27]` (≈ 2 s/instruction) for clear observation; Task C uses `clk_div[24]` (≈ 3 Hz) for a fluid demonstration.

2. Task B – Three New Instructions

We extended the base ISA (R-type, I-type ALU, LW, SW, BEQ) with three new instruction types: **LUI**, **XORI**, and **JAL**. These were chosen to cover U-type, I-type ALU extension, and J-type formats respectively.

LUI (Load Upper Immediate) – U-Type

LUI loads a 20-bit immediate into the upper 20 bits of the destination register, zeroing the lower 12. We implemented this by adding an `ALUSrcA` mux: when `ALUSrcA=1`, the ALU's first input is forced to zero, and the immediate (already upper-shifted by `immGen`) passes through via `ADD`, producing `imm[31:12] | 12'b0`.

imm[31:12]	rd	opcode
0x12345	x1	0110111

Test: `lui x1, 0x12345` $\rightarrow$ `x1 = 0x12345000`.

XORI (XOR Immediate) – I-Type ALU

XORI performs a bitwise XOR between a register and a sign-extended 12-bit immediate. No new hardware was required beyond the existing I-type ALU path; we added `funct3 = 3'b100` as the XOR case in `alu_control`, producing `ALUControl = 4'b0011`.

Test: `addi x1, x0, 15` followed by `xori x2, x1, 6` $\rightarrow$ `x2 = 15  $\oplus$  6 = 9`.

JAL (Jump and Link) – J-Type

JAL is an unconditional jump that saves the return address (PC+4) in the destination register. This required three hardware modifications:

1. **Control:** Added `Jump` and `JumpReg` output signals to `main_control`. JAL asserts `Jump=1`; JALR asserts `JumpReg=1`.
2. **PC Mux:** Updated `PCSrc` logic to `branch_taken | Jump | JumpReg`. A target mux selects `pc_branch` for JAL/branches or `alu_result` for JALR.
3. **Write-Back Mux:** A link mux selects PC+4 as the register write data when `Jump` or `JumpReg` is asserted, enabling the return address to be saved in `rd`.

We also added JALR (`opcode 1100111`) to `immGen` as an I-type for its 12-bit offset, and fixed a critical bug in `alu_control`: for I-type instructions with negative immediates (e.g., `addi x2, x2, -4`), bit 30 of the instruction is part of the immediate, not `funct7[5]`. An `IsRtype` guard was added so that SUB is only selected for actual R-type instructions.

Test: `jal x5, 8` at PC=0x0C jumps to PC=0x14, skipping the instruction at 0x10. The 7-segment display shows: 0000 $\rightarrow$ 0004 $\rightarrow$ 0008 $\rightarrow$ 000C $\rightarrow$ 0014 (skip visible). LED 11 lights up during JAL execution.

Control Signal Summary

Instr	RegW	ALUSrc	Mem2R	Brch	SrcA	ALUOp	Jump	JumpR	PCSrc
LUI	1	1	0	0	1	00	0	0	0
XORI	1	1	0	0	0	10	0	0	0
JAL	1	0	0	0	0	–	1	0	1
JALR	1	1	0	0	0	–	0	1	1

FPGA LED Mapping (Task B Demo)

All 13 control signals are wired directly to the Basys3 LEDs for real-time observation. The 7-segment display always shows the Program Counter. The following table maps each LED to its signal during the live demo:

LED#	Signal	Meaning	Lights up for
LED0	RegWrite	Write result to register	LUI, XORI, JAL, ADDI
LED1	MemRead	Read from data memory	LW only
LED2	MemWrite	Write to data memory	SW only
LED3	ALUSrc	ALU input B = Immediate	LUI, XORI, ADDI
LED4	MemtoReg	Write-back from memory	LW only
LED5	Branch	Branch instruction active	BEQ, BNE
LED6	ALUSrcA	ALU input A = 0 (for LUI)	LUI only
LED7	ALUOp[0]	ALU operation bit 0	R-type, I-type ALU
LED8	ALUOp[1]	ALU operation bit 1	R-type, I-type ALU
LED9	PCSrc	PC is jumping/branching	JAL, JALR, taken Branch
LED10	Zero	ALU result is zero	BEQ (when equal)
LED11	Jump	JAL instruction executing	JAL only
LED12	JumpReg	JALR instruction executing	JALR only

Table 1: LED-to-control-signal mapping for Part B FPGA demonstration on Basys3.

3. Task C – Sum of N with Procedure Call and Stack

Our Task C program computes $\text{Sum} = \sum_{i=1}^N i = \frac{N(N+1)}{2}$, where N is read from the Basys3 switches via memory-mapped I/O. The program is structured as a **main routine** that calls a **function** (`sum_of_n`) using JAL/JALR with proper stack discipline.

Program Flow

```
# === MAIN ===
addi x2, x0, 252    # SP = 252 (stack pointer init)
lw   x10, 256(x0)   # Read N from switches (MMIO at 0x100)
jal  x1, sum_of_n   # Call function, x1 = return address
sw   x10, 0(x0)     # Store result
beq  x0, x0, 0      # Trap (halt)

# === FUNCTION: sum_of_n(x10=N) -> x10=sum ===
addi x2, x2, -4     # Push: SP -= 4
sw   x1, 0(x2)      # Push: save return address to stack
xori x11, x0, 0     # sum = 0 (uses XORI)
lui  x12, 0         # counter = 0 (uses LUI)
lui  x1, 0x00050    # Demonstrate LUI with non-zero value
beq  x10, x0, done  # Edge case: if N=0, skip loop
loop:
    addi x12, x12, 1 # counter++
    add  x11, x11, x12 # sum += counter
    bne  x12, x10, loop # Repeat until counter == N
done:
    add  x10, x11, x0 # Move sum to return register x10
    lw   x1, 0(x2)   # Pop: restore return address
    addi x2, x2, 4    # Pop: SP += 4
    jalr x0, x1, 0    # Return to caller
```

Key Design Decisions

Stack Usage. The function pushes the return address (`x1`) onto the stack at entry and pops it before returning. This is necessary because `lui x1, 0x00050` inside the function overwrites `x1`, demonstrating that the stack correctly preserves callee state.

Procedure Call Convention. The main program calls `sum_of_n` via `jal x1, 12`, which saves PC+4 in `x1`. The function returns via `jalr x0, x1, 0`, jumping back to the instruction after

the call.

New Instructions in Use. All three Task B instructions appear in the function body: LUI initializes the counter and demonstrates a non-zero upper immediate, XORI initializes the sum accumulator, and JAL/JALR handle the call/return.

Memory-Mapped I/O. The FPGA switches are mapped to address 0x100 (256). When `lw` targets this address, the hardware intercepts the read and returns the 16-bit switch value instead of RAM data. Since `sw[0]` controls the display mode, input N is always even.

Verification

For any switch input N , the expected output is $N(N + 1)/2$ displayed in hexadecimal. The 7-segment display shows intermediate sums updating in real time as the loop executes:

Switches	N	Sum (decimal)	Display (hex)
<code>sw[3]</code>	8	36	0024
<code>sw[3]+sw[2]</code>	12	78	004E
<code>sw[4]</code>	16	136	0088

All results were verified against both simulation (`iverilog/vvp`) and FPGA hardware.

Appendix – Brainstorming Notes

The following are our handwritten planning notes created during the design phase, documenting the reasoning behind our instruction selection and program design.

Sum of N numbers
Program in Riscv

$$N(N+1) = 54m$$

New instructions:-
1. ANE $\rightarrow$ B-type
2. XORI $\rightarrow$ I-type
3. LUI $\rightarrow$ U-type

LUI instruction:-
1. stores the first 20 bits of the value in the upper 20 bits of the register
2. ZI leaves the last 12 bits to zero.

$rd = imm \ll 12$
Basically shifting left 12 times.

Reason:-
Because the program has two main registers.

Result
loop counter

This has to be set to zero because if we don't set it to zero, some garbage value would happen.

How does LUI sets Reg to Zero?
1. LUI to 0 immediate value.
2. So it will take first 20 bits of zero and put them on the 20 bits of register.
3. Now those left 12 lower bits will also be set to zero because LUI automatically sets them to 0.

Code:-

```
Counter = 0
result = 0
N = 10
while (Counter != N)
{
    Count++
    Sum = Sum + Counter
}
[Riscv equivalent]
```

Start
addi main

```
main:
# initialize variables
li t0, 0          # t0 = 0
addi t1, t0, 10   # t1 = 10
li t2, 0          # t2 = 0
addi t3, t2, 0    # t3 = 0
li t4, 0          # t4 = 0
addi t5, t4, 0    # t5 = 0

loop:
addi t2, t2, 1    # counter++
addi t1, t1, t2   # sum = sum + counter
bne t2, t1, loop  # if counter != N, go back to loop

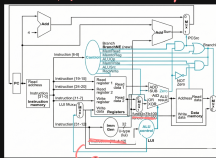
done:
addi t0, t0, t1   # sum result to t0
addi t0, t0, 0
end
```

XORI instruction:-
1. First of all when LUI sets the upper 20 bits = 0
2. The lower 12 are also 0 how so the register is zero
3. But we need to set this register = the value of N
4. That's why we pick those 12-bits and XOR them with N and how our N=10 is correctly initialized.

Noted! XOR anything = anything and we use the same logic for setting the loop counter and result register equal to zero.

Bve:-
why so basically this one is used for running the loop.

Modify Datapath



instruction [31:30]

Figure 1: Design brainstorming: instruction selection rationale, LUI/XORI analysis, pseudocode for the summation loop, and datapath modifications.